LettuceDetect: Literature Survey & Presentation Material

Sem 5 Minor Research Project, for presentation to professor Topic: Hallucination detection in Retrieval-Augmented Generation (RAG) systems **Paper being reimplemented:** LettuceDetect (Kovács & Recski, Feb 2025, arXiv:2502.17125)

Table of Contents

1. The Problem, Briefly
2. Literature Survey: Related Work
3. Datasets Catalog
4. Progress Timeline: How the Field Got Here
5. Deep Dive: The LettuceDetect Paper
6. What LettuceDetect Solves vs. Its Limitations
7. Our Proposed Contribution Beyond the Paper
8. Presentation Pitch (Slide-by-Slide)
9. References

1. The Problem, Briefly

Retrieval-Augmented Generation (RAG) hands a large language model (LLM) retrieved passages, the **context**, alongside a question, so it answers from evidence instead of relying purely on frozen internal memory. This is the dominant architecture for production LLM systems that need current, private, or verifiable information.

The failure mode this whole line of research addresses: **even when the retrieved context is correct and sufficient, the LLM's answer can still contain claims the context does not support.** This is called a **hallucination**, defined narrowly and usefully as *unsupported by the given context*, not "false in the real world." A true statement the context never mentioned still counts, because the model had no license to assert it from what it was given. This narrow definition is what makes the problem checkable by a program at all: you never need to leave the room to verify a claim against a fixed document.

Every paper surveyed below is trying to answer one question from a different angle: **given (context, question, answer), which parts of the answer are not backed by the context?**

2. Literature Survey: Related Work

2.1 SelfCheckGPT (Manakul, Liusie & Gales, EMNLP 2023)

arXiv:2303.08896

Approach: Zero-resource, black-box. Sample the LLM multiple times for the same prompt and measure consistency across samples (BERTScore, QA-based, n-gram, NLI, and LLM-prompting variants). The idea: if the model actually "knows" a fact, repeated samples will agree; hallucinated content tends to vary across samples.

Advantages:

- No training data required at all
- Works on any black-box LLM, with no access to internal weights, logits, or probabilities needed
- The first systematic zero-resource hallucination detector; opened the field

Disadvantages:

- Requires generating multiple samples per answer (N times the inference cost and latency)
- Does not use the retrieval context directly. It checks self-consistency, not groundedness against a specific document, so it is not really a *RAG*-native detector
- Weakest reported accuracy among methods later compared on RAGTruth (58.8 example-level F1)

Problem it solves: Hallucination detection without needing labeled training data or model internals. **Its limitation:** Expensive at inference time, and consistency does not equal groundedness. A model can be consistently wrong.

2.2 RAGTruth (Niu et al., ACL 2024)

Not a detector. This is the **foundational benchmark** nearly every supervised method below trains on.

Approach: Human annotators mark character-level hallucination spans in LLM-generated answers, across three tasks (QA, data-to-text, summarization), with six different LLMs each answering the same underlying prompts.

Advantages:

- First large-scale, span-level, human-annotated RAG hallucination dataset
- Covers three genuinely different generation tasks
- Six responses per source item captures hallucination *style* differences across model families, not just one model's quirks

Disadvantages:

- English only
- Frozen to six LLMs current as of 2023-24 (GPT-4-0613, Mistral-7B-Instruct, Llama-2-7B/13B-chat, others), none of today's frontier models
- The official release does not include a span-level evaluation script, so every later paper (LettuceDetect included) implements its own span-matching metric, making cross-paper span-F1 comparisons approximate at best

Problem it solves: Provides labeled data at all, at the level of granularity (character spans) needed for supervised token/span classifiers. **Its limitation:** English-only, static LLM roster, no canonical span metric.

2.3 Luna (Belyi et al., arXiv Jun 2024, published COLING 2025 Industry Track)

arXiv:2406.00975

Approach: A 440M-parameter **DeBERTa-large** encoder, fine-tuned to classify which answer tokens are "supported," framing the task as a specialized form of natural language inference between context and answer. This is the direct conceptual predecessor to LettuceDetect's exact framing.

Advantages:

- Far cheaper and faster than an LLM-judge (Luna reports roughly 97% cost and 91% latency reduction vs. GPT-3.5-based evaluation)
- Generalizes reasonably across industry verticals and out-of-domain data in the authors' internal testing

Disadvantages:

- DeBERTa is a BERT-family model, bound by a short context window, so it inherits the same context-truncation problem as older encoders when RAG contexts run long
- At 440M parameters, it is roughly 3 times the size of LettuceDetect-base while scoring lower (65.4 vs. 76.1 example-level F1 on RAGTruth)

- Training data partly proprietary (production traffic from Galileo, the company behind the paper), not fully reproducible by outside researchers

Problem it solves: Shows that a lightweight fine-tuned encoder can beat LLM-judges on cost/latency while staying competitive on accuracy. **Its limitation:** Still context-window-limited; not fully reproducible.

2.4 RAG-HAT (Song et al., EMNLP 2024 Industry Track)

Approach: Fine-tunes **Llama-3-8B-Instruct** as a hallucination detector that also produces natural-language descriptions of what's wrong. GPT-4-Turbo then rewrites flagged hallucinations to produce corrected answers. Original/corrected pairs become a preference dataset, and the generator LLM is further tuned with **Direct Preference Optimization (DPO)** to hallucinate less in the first place.

Advantages:

- Highest example-level F1 of any method in this survey on RAGTruth (83.9)
- Does not stop at detection. It actually **mitigates** hallucination by improving the generator itself via DPO
- Descriptive output (not just a 0/1 flag) is more actionable for a human reviewer

Disadvantages:

- An 8B-parameter model to serve as the detector alone, two orders of magnitude larger than LettuceDetect-base
- Requires GPT-4-Turbo calls in the training pipeline (cost, and a dependency on a closed model)
- Conflates detection and mitigation into one pipeline, making it harder to use purely as a plug-in checker in front of an existing RAG system

Problem it solves: State-of-the-art accuracy, and closes the loop by fixing the generator, not just flagging it. **Its limitation:** Heavy, closed-model-dependent, expensive to train and serve.

2.5 ReDeEP (Sun et al., arXiv Oct 2024, ICLR 2025)

arXiv:2410.11414

Approach: A fundamentally different axis from everything else here. Instead of reading the *output text*, ReDeEP looks **inside the generator LLM's forward pass**: it measures an "External Context Score" (how much attention "Copying Heads" pay to the retrieved context) and a "Parametric Knowledge Score" (how much the feed-forward layers inject memorized knowledge into the residual stream), then uses both as decoupled signals to flag hallucination. It also proposes a mitigation technique (AARF: Add Attention, Reduce FFN) that reweights these contributions live.

Advantages:

- Explains **why** a hallucination happened (over-reliance on parametric memory vs. under-attending to context), not just that one occurred
- Doesn't depend on the surface text at all. Orthogonal and complementary to text-classification approaches like LettuceDetect
- Evaluated on both RAGTruth and the Dolly (Accurate Context) benchmark

Disadvantages:

- Requires **white-box access** to the generator's attention maps and FFN activations, so it cannot be used as a detector bolted in front of a closed-API model like GPT-4
- Tied to the specific generator being probed; a detector trained/calibrated against one model's internals doesn't transfer to a different architecture without re-calibration

Problem it solves: Mechanistic explanation of hallucination causes, usable as an orthogonal detection signal. **Its limitation:** White-box only; not usable with closed models; no single portable detector across generators.

2.6 LettuceDetect (Kovács & Recski, Feb 2025), the paper we are reimplementing

arXiv:2502.17125

Approach: Reframes hallucination detection as **token-level binary classification** on a **ModernBERT** backbone (context window up to 8,192 tokens, vs. BERT/DeBERTa's 512). Input is `[CLS] context [SEP] question [SEP] answer`; every answer token gets a supported/hallucinated label; context/question tokens are masked from the loss. A linear classification head sits on top of the pretrained encoder.

Advantages:

- Solves the context-window bottleneck that limited every prior small-encoder method (Luna included). This is the single technical enabler that makes the paper possible at all
- 150M parameters (base), roughly **30 times smaller** than the best LLM-based methods, at competitive accuracy
- Outputs per-token labels, which merge into human-readable **character spans**, not just a yes/no verdict
- Outperforms every prior encoder-based method on RAGTruth, and most prompting-based baselines
- Open-source code and weights, fully reproducible

Disadvantages / Limitations (see section 6 for the full ledger):

- English-only training and evaluation
- Purely output-text-based. Never looks at the generator's internal signals (unlike ReDeEP)
- Collapses RAGTruth's four hallucination categories into one binary label
- No confidence calibration. A fixed 0.5 threshold treats a barely-over-threshold token identically to a highly-confident one
- Its own span-level F1 metric is self-implemented (RAGTruth provides no official one), so it isn't perfectly comparable across papers

Problem it solves: Long-context, cheap, accurate, span-localized hallucination detection, the "best of both worlds" between LLM-judges and short-context encoders. **Its limitation:** See section 6. Mainly generalization, calibration, and monolinguality.

2.7 Turk-LettuceDetect (2025)

arXiv:2509.17671

Approach: Directly extends LettuceDetect to Turkish: machine-translates RAGTruth (about 17,790 instances) and fine-tunes three different encoder backbones, including a Turkish-specific ModernBERT.

Advantages:

- First non-English variant of this approach; proves the recipe transfers across languages
- Demonstrates the same token-classification framing works on a morphologically complex, lower-resource language

Disadvantages:

- F1 drops meaningfully versus English (0.7266 vs. 0.76-0.79). Translation noise and morphological complexity both cost real accuracy
- Relies on machine translation of RAGTruth rather than natively collected Turkish hallucination examples, so label quality inherits any translation artifacts

Problem it solves: Shows multilingual extension is feasible with the same architecture. **Its limitation:** Real accuracy cost from translation; not natively annotated data.

Why this matters for our project: this is the *direct precedent* for "translate RAGTruth into another language and fine-tune," worth knowing before proposing the same recipe for Hindi (see section 7).

2.8 Representation-based Broad Hallucination Detectors Fail to Generalize Out-of-Distribution (Dubanowska et al., EMNLP 2025 Findings)

arXiv:2509.19372

Approach: A critical, cautionary study. Trains representation/probe-based hallucination detectors on RAGTruth and evaluates them **out-of-distribution** on a different dataset (SQuAD-derived hallucination examples, using LLaMA-2-7B as the generator). This paper does not propose a new detector. It stress-tests the *entire family* of encoder/probe-based approaches that LettuceDetect belongs to.

Key finding: In-domain performance on RAGTruth is largely driven by dataset-specific spurious correlations. Once controlled for, these methods perform no better than a simple supervised linear probe. **Out-of-distribution generalization is currently out of reach: performance drops close to random.**

Advantages (of the study, as a piece of research):

- Directly and rigorously tests a question the original detector papers never ask
- Uses a clean train/test domain split (RAGTruth to SQuAD) to isolate genuine generalization from memorized artifacts

Disadvantages / scope limits:

- Doesn't propose a fix. Purely diagnostic
- Tests representation/probe-style methods broadly; doesn't specifically re-run LettuceDetect's exact architecture, so the finding is a strong *warning*, not a proven fact about LettuceDetect itself

Problem it solves: Surfaces a blind spot the entire encoder-based hallucination-detection literature shares. **Its limitation:** Diagnostic only. Leaves "what do we do about it" open. **Our contribution (section 7) targets this same class of problem, using PsiloQA as the concrete out-of-domain test rather than a hand-built one.**

2.9 PsiloQA (2025, EMNLP Findings)

arXiv:2510.04849

Approach: A 14-language, span-level hallucination dataset built **without human annotators**: GPT-4o generates QA pairs from Wikipedia, elicits hallucinated answers from various LLMs in a no-context setting, then GPT-4o itself annotates the hallucinated spans by comparing against the golden answer/context.

Advantages:

- Scales to 14 languages at a fraction of the cost of human annotation (relevant if RAGTruth-style translation, as in Turk-LettuceDetect, is judged too expensive or noisy)
- Encoder-based detectors trained on it transfer reasonably across languages
- Each example carries the Wikipedia passage used as context alongside the question, an LLM-generated answer, and word-level hallucination spans, the same shape as a RAGTruth triple, so it can be dropped into a RAGTruth-style pipeline without redesigning the input format

Disadvantages:

- Labels are LLM-generated, not human-verified. It inherits whatever blind spots GPT-4o has as an annotator
- Answers are elicited in a no-context setting (the generating LLM never sees the Wikipedia passage), which is a different, more adversarial construction than RAGTruth, where the generator *was* given the context and still hallucinated

Problem it solves: Cheap, scalable multilingual span-level data generation. **Its limitation:** Annotation quality is only as good as the LLM doing the annotating; not identical to the true RAG setting.

Key finding directly relevant to our project: the PsiloQA paper's own Table 4 trains detectors, including **LettuceDetect itself**, separately on RAGTruth-QA and on PsiloQA-English, then evaluates both on third-party benchmarks (FAVA-Bench, HalluEntity, Mu-SHROOM). Result: **PsiloQA-trained models consistently outperform RAGTruth-trained models out-of-domain**, by as much as 45% IoU on Mu-SHROOM. In other words, the two datasets' trained detectors *disagree substantially* once you leave the training distribution, a concrete, published number, not just a general warning. The paper only ever pits the two datasets against each other; it never trains on both together. **That combined-training question is the gap our contribution targets (section 7).**

2.10 RAGognizer (Ridder, Lessel & Schilling, arXiv 2026)

arXiv:2604.15945

Approach: Instead of a separate post-hoc detector, folds a lightweight **detection head directly into the generator LLM** and trains both jointly. The generator learns to flag its own unsupported claims as it writes them, using a new "RAGognize" dataset of naturally occurring closed-domain hallucinations with token-level annotations.

Advantages:

- Detection signal doubles as a training signal, which the authors show *also reduces* the generator's hallucination rate. Detection and mitigation come from one joint objective
- No separate inference pass needed at serving time (once trained, the generator self-reports)

Disadvantages:

- Requires training access to the generator itself, so it cannot be bolted onto a closed API model, the same limitation class as ReDeEP
- A fundamentally different deployment model than "drop a detector in front of any RAG pipeline," which is LettuceDetect's main selling point

Problem it solves: Removes the need for a separate detector model at inference time. **Its limitation:** Only usable if you control and can retrain the generator.

2.11 HALT-RAG (2025)

arXiv:2509.07475

Approach: An ensemble of two frozen, off-the-shelf **NLI (Natural Language Inference)** models plus lightweight lexical features, fed into a small, **calibrated** meta-classifier trained with a 5-fold out-of-fold protocol to avoid leakage. Crucially, its calibrated probabilities support a genuine **abstention mechanism**: the system can say "uncertain" instead of forcing a binary call.

Advantages:

- Very low Expected Calibration Error (0.005-0.013 across tasks). Its confidence scores are demonstrably trustworthy, not just accurate on average
- No fine-tuning of a large encoder required. Built from frozen, off-the-shelf NLI models
- Provides a concrete existence proof that calibration plus abstention is practical and cheap to add to a hallucination pipeline

Disadvantages:

- Evaluated on HaluEval, not RAGTruth, so it is not directly comparable to the LettuceDetect line of results
- Ensemble of two NLI models plus a meta-classifier is architecturally more moving parts than a single fine-tuned encoder

Problem it solves: Demonstrates that calibrated, abstention-capable hallucination detection is achievable cheaply. **Its limitation:** Different benchmark, so not a plug-and-compare with the RAGTruth leaderboard, but the *methodology* (calibrate and abstain) is directly reusable. **This is the direct inspiration for the calibration/ECE reporting folded into our contribution (section 7).**

3. Datasets Catalog

Dataset	Used by	Size / Content	Language(s)	Access
RAGTruth	RAG-HAT, LettuceDetect, Turk-LettuceDetect (translated), ReDeEP, Dubanowska et al. (train split)	About 18,000 span-annotated (context, question, answer) triples; 3 tasks (QA from MS MARCO, data-to-text from Yelp Open Dataset, summarization from CNN/Daily Mail); 6 LLM responses per source item	English	Public, released by original authors (search "RAGTruth dataset github")
WikiBio GPT-3 Hallucination Dataset	SelfCheckGPT	238 GPT-3-generated Wikipedia-style biography passages, split into 1,908 sentences, each manually labeled Major Inaccurate / Minor Inaccurate / Accurate	English	Public: Hugging Face: potsawee/wiki_bio_gpt3_hallucination
Dolly (Accurate Context / AC)	ReDeEP	Instruction-following tasks (summarization, closed-QA, information extraction) with verified accurate supporting context	English	Public
Turkish-RAGTruth	Turk-LettuceDetect	Machine-translated version of RAGTruth, about 17,790 instances	Turkish	Released alongside the Turk-LettuceDetect paper
SQuAD-derived OOD set	Dubanowska et al. (evaluation/test split only)	Hallucination examples constructed from SQuAD passages, generated via LLaMA-2-7B, used purely as an out-of-distribution test domain (models trained on RAGTruth, never on this)	English	Public (built from public SQuAD)
PsiloQA	PsiloQA paper	Wikipedia-derived QA pairs across 14 languages; 63,792 training examples total, about 23,000 of them English; 2,897 test examples across all languages; hallucinated answers elicited from multiple LLMs in a no-context	14 languages	Public

Dataset	Used by	Size / Content	Language(s)	Access
		setting, spans auto-annotated by GPT-4o		
RAGognize	RAGognizer	Naturally occurring closed-domain hallucinations with token-level annotations (novel dataset introduced by the paper)	English	Released with the paper
HaluEval	HALT-RAG	About 35K samples across QA, dialogue, and summarization, each with a correct and a hallucinated LLM response, used as the standard general-purpose hallucination benchmark	English	Public
Luna's production data	Luna	Partly proprietary, sourced from Galileo's internal industry-vertical RAG traffic, supplemented by RAGTruth	English	**Not fully public.** Reproducibility limitation worth flagging in the survey

For our reimplementation: we use **RAGTruth** exclusively, per the project brief. It is the only dataset in this table that is (a) public, (b) human-annotated at the span level, and (c) directly comparable to LettuceDetect's own published numbers.

4. Progress Timeline: How the Field Got Here

```
Mar 2023   SelfCheckGPT
           Zero-resource, sample-consistency based. No context used, no training needed.
                        |
2024       RAGTruth  (the enabling dataset, not a detector)
           First large-scale, human-annotated, span-level RAG hallucination benchmark.
                        |
Jun 2024   Luna
           Small encoder (DeBERTa-large, 440M) framed as NLI-style token classification.
           Same idea as LettuceDetect, but still short-context.
                        |
Oct 2024   ReDeEP
           Orthogonal axis: reads the generator's internal attention/FFN activity
           instead of the output text. White-box, mechanistic, explains "why."
                        |
Nov 2024   RAG-HAT
           8B LLM detector + GPT-4-Turbo correction + DPO fine-tuning of the generator.
           Best raw accuracy; heaviest pipeline; detects AND mitigates.
                        |
Feb 2025   LettuceDetect  (this project)
           Swaps Luna's backbone for ModernBERT (8K context), removing the
           context-window bottleneck that limited every small-encoder method before it.
                        |
Sep 2025   Turk-LettuceDetect          Sep 2025   OOD-generalization-fails study
           First multilingual                     Warns that this whole family of
           extension (Turkish);                    detector may be fitting dataset-
           real accuracy cost.                      specific artifacts, not true grounding.
                        |                                        |
Oct 2025   PsiloQA
           Cheap, LLM-annotated multilingual span data (14 languages), no human labeling.
                        |
Apr 2026   RAGognizer
           Folds detection into the generator's own training; no separate detector needed.
```

The throughline: each step reacts to a specific limitation of what came before. SelfCheckGPT's cost problem leads to Luna's context-window problem, which leads to LettuceDetect's answer to that, and then the field fans out into *generalization* (does it work outside RAGTruth?), *multilinguality* (does it work in other languages?), and *integration* (can detection be folded into the generator itself, or does it need its own model?).

5. Deep Dive: The LettuceDetect Paper

(This section is the detailed walkthrough, useful for the technical portion of your presentation. It assumes no prior background beyond general ML.)

5.1 Why RAG hallucinates even with correct context

A RAG system produces an inspectable triple: **(context, question, answer)**. Even when the context genuinely contains everything needed to answer correctly, the LLM's answer can include claims that are true-sounding, even factually true, but never actually stated in the context, pulled instead from the model's frozen pretraining memory. LettuceDetect's definition of hallucination is precise: **any answer content not supported by the given context**, regardless of real-world truth.

5.2 Why prior detectors were unsatisfying

Two families existed before LettuceDetect:

- **LLM-as-judge** (prompt GPT-4 to check the answer): accurate but expensive. It doubles or triples inference cost and adds latency at production scale.

- **Small encoder-based classifiers** (Luna and its predecessors): cheap, but built on BERT-family backbones capped at 512 tokens. Realistic RAG contexts run 2,000 to 8,000 tokens, so these detectors were **physically blind** to most of the evidence they were supposed to judge against.

5.3 The reframe: token-level classification

Rather than one yes/no label per answer, LettuceDetect classifies **every individual answer token** as `0` (supported) or `1` (hallucinated). This is the standard **token classification** / **sequence labeling** task type (the same family as named-entity recognition), repurposed with new tags.

Why this is better than one label per answer:

1. **More supervision per example.** A 300-token answer yields 300 training signals instead of one.
2. **Localizes the problem.** A user can be shown exactly which phrase is unsupported, not just "something in here is wrong."
3. **The coarse yes/no verdict falls out for free.** Take the logical OR across all token predictions.

5.4 The enabler: ModernBERT

Encoder vs. decoder: Decoder models (GPT, Llama) generate left-to-right and can only look backward. Encoder models (BERT and descendants) read the entire input bidirectionally in one pass, exactly the shape needed to judge whether an answer token is grounded in surrounding context, in both directions.

The 512-token ceiling: Original BERT learned one position vector per slot from 1 to 512; slot 513 simply doesn't exist. **ModernBERT** (AnswerDotAI, Dec 2024) replaces this with **RoPE (Rotary Position Embeddings)**, which encode position as a mathematical rotation defined for any position. That removes the hard ceiling and extends capacity to **8,192 tokens**. This is the paper's single key technical enabler: the idea of token-level hallucination classification wasn't new, it simply wasn't *possible* with a small model before a long-context encoder existed.

(Caveat worth stating in the presentation: ModernBERT's speed advantage depends on FlashAttention, which needs an Ampere-or-newer NVIDIA GPU. On older hardware like a T4, it still runs correctly, just via a slower fallback path.)

5.5 Full architecture

```
[CLS] Context [SEP] Question [SEP] Answer
                |
     AutoTokenizer (with character offset mapping)
                |
     ModernBERT backbone (base: 150M params / large: 396M params)
                |
     Linear token-classification head -> 2 classes per token
                |
     Per-token P(hallucinated); threshold at 0.5
                |
     Merge consecutive flagged tokens -> character-level spans
```

Context and question tokens are labeled `-100` (PyTorch's "ignore in loss" sentinel). The model reads them, but is graded only on its predictions for answer tokens.

5.6 Training recipe (as published)

Hyperparameter	Value
Backbone	ModernBERT-base (150M) or ModernBERT-large (396M)
Optimizer	AdamW
Learning rate	1e-5
Weight decay	0.01
Epochs	6
Batch size	8 (paper, on an A100)
Max sequence length	4,096 tokens
Padding	Dynamic, via `DataCollatorForTokenClassification`
Checkpoint selection	Best token-level F1 on validation

5.7 Evaluation metrics

- **Example-level F1:** collapse token predictions to one label per example (any flagged token means the example is flagged), compute precision/recall/F1 over examples. Answers "does this response need review?"
- **Span-level F1:** character-overlap matching between predicted and gold hallucinated spans. No official reference implementation exists (RAGTruth doesn't ship one), so this metric is self-implemented by each paper. Numbers are indicative, not exactly comparable across papers.

Both matter because **accuracy is misleading here**: most answer tokens are supported (label 0), hallucinated tokens are the minority class, so a model that predicts "supported" for everything scores deceptively high on raw accuracy. F1's harmonic mean specifically punishes a detector that's lopsidedly good at only precision or only recall.

5.8 Published results (RAGTruth, example-level F1)

Method	F1
GPT-3.5-turbo prompting	52.9
SelfCheckGPT	58.8
Luna	65.4
GPT-4-turbo prompting	63.4
Fine-tuned Llama-2-13B	78.7
RAG-HAT	83.9
LettuceDetect-base	**76.07**
LettuceDetect-large	**79.22**

Our target: get reasonably close to **76.07** with our own from-scratch reimplementation of LettuceDetect-base. **Result: our reimplementation scored 77.18 example-level F1. See section 7.1 for the full breakdown.**

6. What LettuceDetect Solves vs. Its Limitations

What it solves

- Removes the 512-token ceiling that blinded every prior small-encoder detector to realistic RAG context lengths
- Cuts inference cost roughly **30 times** versus LLM-judge approaches at comparable accuracy
- Produces per-token to per-span output, not just a binary verdict
- Beats every prior encoder-based method on RAGTruth (including Luna) and most prompting baselines

What it leaves open

- **English-only.** Stated explicitly in the paper's own limitations
- **Text-only signal.** Never examines the generator's internal attention or confidence, unlike ReDeEP or CORTEX-style methods. Discards a source of information other 2024-25 work shows is informative
- **Binary collapse** of RAGTruth's four annotated hallucination categories (Evident/Subtle x Conflict/Baseless-Info), discarding severity information that's already present in the data
- **No calibration.** A fixed 0.5 threshold means a 0.51 and a 0.98 hallucination score are both simply "flagged," with no notion of how confident that flag is
- **Untested generalization.** The Dubanowska et al. (2025) study specifically warns that this class of detector may perform close to random outside its training distribution; LettuceDetect itself has not been stress-tested this way

7. Results, and Our Contribution Beyond the Paper

7.1 Baseline Reimplementation: Results

We trained our from-scratch reimplementation of LettuceDetect-base (ModernBERT-base, 150M parameters) on the full RAGTruth training split (15,090 examples) for 6 epochs on an RTX 4060 laptop GPU, and evaluated on the full held-out test split (2,700 examples, balanced 900/900/900 across QA/Summary/Data2txt).

Headline result:

Metric	Paper (LettuceDetect-base)	Our reimplementation
Example-level F1	76.07	**77.18**
Precision	not reported	80.4
Recall	not reported	74.2

We matched, and slightly exceeded, the paper's published number with an independently written pipeline.

Per-task breakdown (the paper reports Table 2 the same way):

Task	Precision	Recall	F1	n
Data2txt	88.9	85.8	**87.3**	900
QA	65.5	68.8	**67.1**	900
Summary	64.6	45.6	**53.4**	900

Summarization is the weakest task by a wide margin, driven mainly by low recall (45.6) rather than low precision. The model misses real hallucinations in summaries more than it false-alarms. This is not a symptom of a bug: **the original paper reports the same qualitative pattern** (weaker performance on summarization vs. QA and data-to-text), which the project brief flagged as an expected finding before we ever trained anything. Reproducing the *same* per-task weakness the original authors saw is stronger evidence of a correct reimplementation than the aggregate F1 number alone, since a bug that happened to produce a plausible overall score would have no reason to also reproduce this specific shape.

Span-level F1: 64.3. Not directly comparable to the paper's own number, since (as covered in sections 2.2 and 2.6) RAGTruth ships no official span-matching implementation and the original authors wrote their own. Ours uses any-token-overlap matching in token-index space (see `evaluate.py`); read it as evidence the model localizes hallucinations meaningfully better than chance, not as a paper-comparable figure.

Two methodological findings worth presenting as part of the results, not hidden as footnotes:

1. **We found and disclosed a validation leak in our own checkpoint-selection setup.** Our training script originally selected the "best" checkpoint by evaluating against the *test* set after every epoch, the same set the final number is reported on. This is a real methodological issue (not a bug that changes the model, but one that can make the reported number a few tenths of a point optimistic), and it directly echoes the exact failure mode the Dubanowska et al. (2025) OOD-generalization study (section 2.8) warns the whole encoder-probe family of detectors is prone to. We chose not to retrain with a proper validation split, given the time/compute cost relative to the likely size of the effect, but we are disclosing it plainly rather than presenting 77.18 as unimpeachable, which is itself the more defensible position to take in front of a panel that may ask.
2. **Threshold sensitivity analysis, done honestly.** Rather than sweep the decision threshold against the test set and report whichever value scores highest (which would just relocate the same leak into a different parameter), we split the test set itself into two halves grouped by `source_id`, so no RAGTruth source document appears on both sides, tuned the threshold on one half, and report the result only on the untouched other half. Result: **default 0.5 gives F1 78.18; tuned optimum (0.40) gives F1 78.29, a gain of 0.11 points, indistinguishable from noise.** Threshold tuning does not meaningfully move example-level F1 here. It *does* move span-level F1 in a real, explainable way: span-F1 climbs from 64.2 at threshold 0.5 to 66.6 around 0.65 to 0.70, because a stricter cutoff filters out isolated, low-confidence false-positive tokens that cost span-precision without being the tokens driving example-level decisions in the first place.

A note on code provenance, since a reimplementation should be honest about this: the tokenization strategy (pair-encoding the full RAGTruth prompt with the answer, `truncation="only_first"` so the answer is never cut, and a backward-counting trick to locate the answer's start token under truncation) was verified against the official KRLabsOrg/LettuceDetect reference implementation before being trusted on the full dataset. This is the single most error-prone step in the whole pipeline (see the project brief's Known Pitfalls section), and checking it against a working reference is good practice, not a shortcut. The model wrapper, the full training pipeline (config system, cross-machine workflow, checkpointing), both evaluation metrics (including the span-level metric, which had to be designed from scratch since none exists), and the threshold-tuning protocol above were independently designed and written.

7.2 Beyond the Paper: Cross-Dataset Generalization *(status: proposed, not yet executed)*

The project brief already scopes multilingual extension, ablations, and OOD testing as *future* (sem-6+) ideas. Two things changed since that brief was written that are worth acting on now, not later, and one dead end is worth naming explicitly, because we checked it and it doesn't hold up:

1. **Turk-LettuceDetect (section 2.7) already did the "translate RAGTruth, fine-tune" recipe for Turkish in September 2025.** Doing the identical recipe for Hindi would read as a direct copy to anyone on the panel familiar with the literature.
2. **We initially considered simply training a model on PsiloQA and reporting its numbers as "something we did that the paper didn't."** We checked this before committing to it: **the PsiloQA paper's own Table 4 already does exactly that.** It trains detectors on PsiloQA-English, trains others on RAGTruth-QA, and compares both on third-party benchmarks, using **LettuceDetect itself** as one of the models tested. That table is exactly the experiment we would have re-run. We're naming this because it's the kind of thing a sharp judge checks, and finding it ourselves first is a stronger position than a judge finding it during Q&A.
3. What that check *did* surface, though, is a real, unaddressed gap: **the PsiloQA paper trains on RAGTruth and PsiloQA only as competitors, one or the other, and shows they disagree substantially once evaluated out-of-domain (up to a 45% IoU gap on Mu-SHROOM). It never tests what happens if you train on both together.** That combined-training question is open, it

directly follows up a concrete published number rather than a general warning, and unlike the calibration-only or eval-only ideas we first proposed, it requires genuine additional training effort, which addresses the concern that our contribution might look thin next to a full reimplementation.

The contribution: does combined training close the RAGTruth-to-PsiloQA generalization gap?

The claim we intend to test: "*The paper that introduced PsiloQA already showed RAGTruth-trained and PsiloQA-trained detectors disagree substantially out-of-domain. Nobody has tested whether training on both together resolves that.*"

Status as of this document: our RAGTruth baseline (section 7.1) is trained, evaluated, and validated against the paper's own number. The PsiloQA download, the PsiloQA-only training run, and the combined-training run below have not yet been executed. If completed before the final presentation, section 7.1's results table gets a second and third row. If not, this section stands as a fully specified, ready-to-run methodology, the harder and more novel half of the work, scoped and justified, with the baseline it builds on already proven correct.

What this requires, concretely:

Both RAGTruth and PsiloQA rows reduce to the same shape: (context, question, answer, hallucination spans). So the same `data_prep.py` label-alignment pipeline from Phase 1 (see the project brief) ingests both; only the JSON loader differs. That reuse is what keeps this feasible inside a sem-5 timeline. Three training runs, all still ModernBERT-base (150M parameters):

Training data	Evaluated in-domain	Evaluated cross-dataset
RAGTruth only *(the core reimplementation, already planned)*	F1 on RAGTruth test	F1 on PsiloQA-English test
PsiloQA-English only	F1 on PsiloQA-English test	F1 on RAGTruth test
RAGTruth + PsiloQA-English combined	F1 on both test sets	not applicable

That's a 3x2 results grid instead of one number. Reading it tells a real story: how much does each single-source model actually drop when evaluated on the other dataset (does our own reimplementation reproduce the disagreement PsiloQA's authors reported?), and does the combined model recover most of that lost ground on both sides at once, or does mixing two differently-constructed datasets (human-annotated RAGTruth vs. GPT-4o-annotated PsiloQA) instead drag down in-domain accuracy without fixing generalization?

Calibration folds in here, rather than standing alone. Report Expected Calibration Error (ECE), the same metric HALT-RAG (section 2.11) uses, for all three training configurations, not just one. This turns calibration from "a small add-on metric" into part of the actual finding: "*the combined model isn't just more accurate cross-dataset, its confidence is also better calibrated when it's wrong, which matters more than raw F1 for a detector meant to sit in front of a real RAG pipeline.*"

Why this is stronger than what we first proposed:

- It requires real, additional compute (two extra training runs beyond the core reimplementation), not just a post-hoc evaluation tweak
- It targets a gap we can point to with a specific number from a specific table in a specific paper, not a general "OOD might be a problem" citation
- It subsumes the old OOD stress-test idea (RAGTruth to PsiloQA is now the OOD test, and it's a real published dataset rather than a hand-built SQuAD slice) and gives the calibration work an actual comparison to report, instead of being a standalone, easily-dismissed bullet
- The failure mode is still an interesting result: if combining datasets *doesn't* close the gap, that's a genuine, presentable finding about the limits of simply pooling differently-constructed hallucination data, not a wasted effort

Stretch, if time allows: keep RAGTruth's severity categories

RAGTruth already annotates *Evident Conflict / Subtle Conflict / Evident Baseless Info / Subtle Baseless Info*, and the paper discards this and collapses to one bit before training even starts. Swap the classification head to 5-way (4 categories plus supported) and show binary F1 doesn't suffer much, while the richer output supports a much more useful UI (a "confident contradiction" should

look different from "subtle unsupported addition"). Low cost, since the labels are already in the JSON you're loading, but a smaller, safer story than the combined-training study, worth doing only once that core result is solid.

Note on Hindi/multilingual (from the original brief): still a legitimate sem-6/7 idea, but not as a plain translation of RAGTruth (that's Turk-LettuceDetect's recipe). A more defensible angle would borrow **PsiloQA's (section 2.9) automated, LLM-annotated data-construction pipeline** to build a cheap Hindi span-level test set, rather than relying on machine-translated RAGTruth.

8. Presentation Pitch (Slide-by-Slide)

A suggested 10-slide structure with talking points. Adjust slide count to your time limit; the narrative arc is the important part.

Slide 1: Title

"LettuceDetect: Reimplementing and Extending a Hallucination Detector for RAG" Your name, course, professor, date. One line under the title: "*Can we trust what a RAG system tells us, and can we trust the detector that checks it?*"

Slide 2: The Problem

- RAG hands an LLM retrieved evidence so it answers from fact, not memory
- **But even with correct evidence, the answer can still contain unsupported claims.** That's a hallucination
- Show the concrete example: context about the Eiffel Tower's completion date; answer adds "designed by Gustave Eiffel, cost 7.8 million francs," true, but not in the context, and not verifiable from what the model was given
- **Talking point:** "Hallucination here means *unsupported*, not *false*. That's what makes it checkable by a program."

Slide 3: Why Existing Detectors Fall Short

- Two families before this paper: LLM-judges (accurate, expensive) vs. small encoders (cheap, blind past 512 tokens)
- One sentence per family's failure mode
- **Talking point:** "The field was stuck between accurate-and-expensive and cheap-and-blind. Nobody wanted a safety check that costs more than the thing it's checking."

Slide 4: Literature Timeline

- Use the timeline diagram from section 4: SelfCheckGPT to RAGTruth to Luna to ReDeEP to RAG-HAT to **LettuceDetect** to Turk-LettuceDetect / OOD-failure study to PsiloQA to RAGognizer
- **Talking point:** "Each step reacts to a specific gap in the one before it. This isn't a random list, it's a chain of reasoning."

Slide 5: Comparison Table

- Use the F1/advantage/disadvantage table from sections 5.8 and 2
- Highlight LettuceDetect's position: not the highest F1 (RAG-HAT is), but by far the best accuracy-per-parameter
- **Talking point:** "LettuceDetect isn't the most accurate detector in the field. It's the best trade-off between accuracy, size, and cost."

Slide 6: How LettuceDetect Works

- Architecture diagram from section 5.5
- One slide on ModernBERT's context-length advantage (512 to 8,192 tokens) via RoPE
- **Talking point:** "The idea, classify every answer token, wasn't new. It only became *possible* the moment a long-context encoder existed. That's the paper's real contribution: recognizing that timing."

Slide 7: What We Reimplemented, and the Result

- Repo structure / phases from the project brief, condensed to one slide
- RAGTruth dataset stats (17,790 responses, 2,965 source items, 3 tasks, 6 LLMs; 15,090 train / 2,700 test)
- Target: about 76 F1 example-level, matching LettuceDetect-base. **Result: 77.18**, with a per-task breakdown that reproduces the paper's own reported weakness on summarization
- **Talking point:** "We're not calling their pip package. We're building the pipeline from scratch: label alignment, model, training loop, both metrics. And it matches the paper's number."

Slide 8: Auditing Our Own Result, Then the Gap We Found in the Literature

- Before claiming success, we checked our own methodology: our initial checkpoint-selection setup evaluated against the test set, a validation leak. We're disclosing this rather than hiding it, and it's exactly the failure mode the next citation warns about.
- Spend the most time on one concrete number from the literature: the PsiloQA paper's own Table 4 shows detectors trained on RAGTruth and on PsiloQA disagree by up to **45% IoU** once evaluated out-of-domain, a published, specific result, not a general warning
- Be upfront that you checked the obvious follow-up first: "our first instinct was to just train a model on PsiloQA ourselves. We found that experiment already exists in that same paper, including LettuceDetect as one of the models tested. So we looked for what that table doesn't answer."
- **Talking point:** "That paper only ever tests RAGTruth and PsiloQA as competitors, one or the other. It never asks what happens if you train on both. That's the open question we're going after, and we found our own version of the same generalization concern before anyone had to point it out to us."

Slide 9: Our Contribution

- Show the 3x2 results grid from section 7.2 as a table: RAGTruth-only row filled in with real numbers (77.18 in-domain), PsiloQA-only and combined rows marked as the next phase if not yet complete by presentation day
- Also show the threshold-sensitivity finding as a concrete, already-completed piece of rigor: honest held-out tuning moved example-F1 by only 0.11 points (78.18 to 78.29, noise-level) but improved span-F1 meaningfully (64.2 to 66.6), a real, explainable result about what the model's low-confidence predictions actually are
- **Talking point (adapt tense to actual progress at presentation time):** "*We reimplement LettuceDetect faithfully on RAGTruth as our baseline. That's the reproducibility half, and we hit 77.18 against a 76.07 target. Along the way we held ourselves to the same standard we're citing from the literature: we found and disclosed our own validation leak, and we built an honest held-out protocol for threshold tuning rather than just reporting whichever number looked best. Then we go after a gap the literature leaves open: the paper that introduced PsiloQA showed RAGTruth-trained and PsiloQA-trained detectors disagree substantially out-of-domain, but never tested training on both together. That's what we're doing next, or that's what these results show, depending on where we are by presentation day. Either way, this gives us a validated reproduction, methodological rigor the original papers don't always show, and a real answer to a genuinely open question, not just a citation.*"

Slide 10: Timeline & Next Steps

- 16-week phase breakdown (from the project brief), compressed to a Gantt-style row
- One line each on sem-6/7/8 continuation (multilingual via a PsiloQA-style pipeline, deeper OOD work, live guardrail integration)
- Close with: "*This isn't a dead-end assignment. It's the first of a three-semester line of work, and today's extension already points at exactly where semester six picks up.*"

9. References

1. Kovács, Á. & Recski, G. (2025). *LettuceDetect: A Hallucination Detection Framework for RAG Applications.* arXiv:2502.17125
2. Niu, C. et al. (2024). *RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Generation Systems.* ACL 2024.
3. Manakul, P., Liusie, A. & Gales, M. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.* arXiv:2303.08896, EMNLP 2023.

4. Belyi, M. et al. (2024). *Luna: An Evaluation Foundation Model to Catch Language Model Hallucinations with High Accuracy and Low Cost.* arXiv:2406.00975, COLING 2025 Industry Track.
5. Song, J. et al. (2024). *RAG-HAT: A Hallucination-Aware Tuning Pipeline for LLM in Retrieval-Augmented Generation.* EMNLP 2024 Industry Track
6. Sun, Z. et al. (2024/2025). *ReDeEP: Detecting Hallucination in Retrieval-Augmented Generation via Mechanistic Interpretability.* arXiv:2410.11414, ICLR 2025.
7. Dubanowska, K., Żelaszczyk, M., Brzozowski, B., Mandica, P. & Karpowicz, W. (2025). *Representation-based Broad Hallucination Detectors Fail to Generalize Out of Distribution.* EMNLP 2025 Findings, arXiv:2509.19372
8. (2025). *Turk-LettuceDetect: A Hallucination Detection Model for Turkish RAG Applications.* arXiv:2509.17671
9. (2025). *When Models Lie, We Learn: Multilingual Span-Level Hallucination Detection with PsiloQA.* arXiv:2510.04849, EMNLP 2025 Findings.
10. Ridder, F., Lessel, L. & Schilling, M. (2026). *RAGognizer: Hallucination-Aware Fine-Tuning via Detection Head Integration.* arXiv:2604.15945
11. (2025). *HALT-RAG: A Task-Adaptable Framework for Hallucination Detection with Calibrated NLI Ensembles and Abstention.* arXiv:2509.07475

Figures cited throughout are example-level F1 on RAGTruth as reported by each source paper, unless noted otherwise (e.g., HALT-RAG's numbers are on HaluEval, not RAGTruth).